CMSC 330 Programming Languages

Homework: AI-Guided Practice in OCaml and Grammars

For this homework, use **ChatGPT, Claude, Gemini, or Grok** as a practice tutor. You will give the AI the **in-class examples printed in this assignment**, ask it to create new exercises at a similar level, solve those exercises yourself, and then ask the AI for feedback.

Submit the complete chat history to D2L. The history must show the exercises generated by AI, your own attempt before seeing the answer, the AI feedback, and any corrections you make.

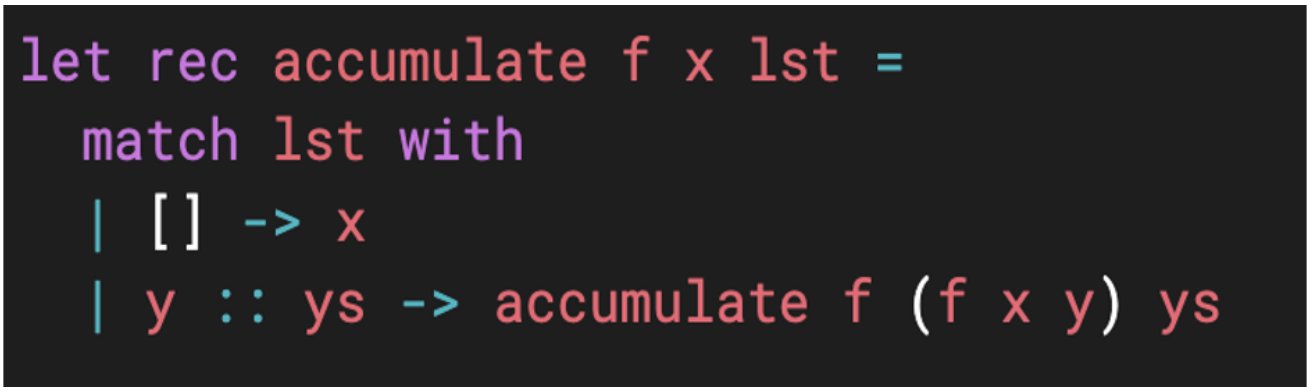
Required Workflow

1. Copy or describe the required **in-class example(s)** from this assignment into your AI conversation.
2. Ask the AI to create new exercises that practice the same ideas and are at about the same difficulty.
3. Tell the AI: **Do not give me the answers yet.**
4. Solve each exercise yourself and send your answer to the AI.
5. Ask the AI to check your answer and explain any mistakes.
6. If needed, revise your answer yourself and ask the AI to check it again.
7. Submit the complete conversation to D2L.

Part 1. OCaml Types of Functions

This part focuses on inferring the types of OCaml functions, especially higher-order functions and curried functions.

In-Class Example 1 - Use this example in your AI prompt



```
let rec accumulate f x lst =
  match lst with
  | [] -> x
  | y :: ys -> accumulate f (f x y) ys
```

```
let rec accumulate f x lst = match lst with | [] -> x | y :: ys -> accumulate f (f x y) ys
```

Questions discussed in class:
1. What is the type of `f`? **Note that x and y may not have the same type.**
2. What is the type of `accumulate`?
3. What does `accumulate` do?

Your task

Show this in-class example to the AI and ask it to create **5 new type-inference exercises** at a similar level. The new exercises should include recursion, lists, currying, and higher-order functions. At least two should require careful reasoning about type variables, including cases where arguments do not have to share the same type.

For every generated exercise, first determine the function's type yourself. Also explain briefly how the type is constrained by the body of the function and what the function does. Then ask the AI to check your work.

Part 2. Writing OCaml Functions

Ask the AI to create **6 new OCaml programming exercises** for you to solve from scratch. The focus is on the material practiced in class: basic function writing, recursion, currying, and tail recursion. **Do not use tuples yet.**

- **2 basic recursive functions**
- **2 curried functions**
- **2 tail-recursive functions** that use a helper function and accumulator

The exercises may use integers, floats, booleans, strings, and lists. They should not rely on tuples, records, objects, references, or advanced library functions that avoid the recursion you are supposed to practice.

For each exercise, write the OCaml function yourself first. Then send your code to the AI and ask it to check correctness, recursion style, currying, and whether a tail-recursive solution is actually tail recursive when required.

Part 3. CFG, BNF, and EBNF

This part focuses on programming-language grammars, including CFG design, precedence, associativity, BNF, EBNF, and ambiguity. Use the following **in-class examples** in your AI prompt.

In-Class Example 2 - Expression Grammar

Write unambiguous rules for expressions with +, /, and ().
Precedence is: **()**, then **/**, then **+**.

Add a rule to our <expr> grammar for exponentiation (**), with precedence one higher than division (/).
This operator should be **right associative**.

In-Class Example 3 - BNF to EBNF

```
<expr> -> <expr> + <term> | <expr> - <term> | <term> <term> -> <term> * <factor> | <term>
/ <factor> | <factor>
```

In-class question: What does the EBNF look like?

Your task

Give the AI the two grammar examples above and ask it to create **6 new exercises**. The exercises must cover all of the following:

1. Design a CFG for a small piece of programming-language syntax.

2. Write an unambiguous expression grammar with specified operator precedence.

3. Add an operator to an existing expression grammar with a specified precedence and associativity.

4. Convert BNF to EBNF.

5. Determine whether a CFG is ambiguous.

6. Show that a CFG is ambiguous by giving two different parse trees or two different derivations for the same string.

Do not ask the AI for the solution until after you have attempted each exercise. For grammar problems, your explanation should make clear how your grammar enforces precedence and associativity, or why a grammar is ambiguous.

Suggested Prompt Structure

*"I am practicing for CMSC 330 Programming Languages. Below are examples we did in class. Use them only as examples of the topics and difficulty. Create new exercises for me, but **do not give me the answers**. I will solve each exercise first. After I send my answer, check it, explain any mistake, and let me revise it."*

What to Submit to D2L

1. The **complete AI chat history** for Parts 1-3.

2. The in-class examples you gave to the AI.

3. All exercises generated by the AI.

4. Your first attempt at every exercise.

5. The AI's feedback on each attempt.

6. Your corrected/revised work when your first answer was wrong or incomplete.

The purpose of this assignment is practice. Your chat history should show that you attempted the problems before asking the AI for solutions.